LABORATORIO SEMANAL

Acceso a datos y visualización en TableView

Catálogo de productos de la papelería Campus

Valor	Modalidad propuesta	Fecha y hora de entrega
30 puntos	Individual	Por definir por el docente

Estudiante: _____ Grupo: _____

1. Situación del problema

La papelería Campus registra sus productos en una hoja de cálculo. El personal necesita consultar precios y existencias, localizar productos sin revisar todo el listado y evitar códigos repetidos o cantidades inválidas. Actualmente, los cambios no siempre se reflejan en el listado que se consulta.

Desarrolle una aplicación de escritorio con JavaFX que almacene el catálogo en PostgreSQL y acceda a él mediante JDBC. La información deberá mostrarse en un TableView y mantenerse coherente con la base de datos al consultar, crear, actualizar y eliminar productos.

2. Indicadores de logro

IL2.2. Implementa operaciones CRUD mediante sentencias SQL y controles de acceso a datos.

IL2.3. Valida y gestiona la información almacenada garantizando su consistencia.

3. Producto que debe entregar

- Una interfaz con TableView, formulario de productos, búsqueda por nombre o código y filtros por categoría y existencias.
- Un módulo CRUD conectado a PostgreSQL, con validaciones, confirmación de eliminación y manejo de errores; los cambios deben persistir al reiniciar la aplicación.
- Proyecto fuente, scripts SQL, README, evidencias de pruebas y repositorio GitHub accesible al docente. Los detalles de entrega están en la página 7.

4. Organización del trabajo

Sesión	Avance esperado
1	Listado desde PostgreSQL, búsqueda y filtros parametrizados.
2	Validaciones, restricciones y tratamiento de excepciones.
3	Integración del CRUD, pruebas completas y documentación.

Alcance: una sola entidad, Producto. No se exigen inicio de sesión, ventas, facturación, instalador ni reportes adicionales. Puede usar FXML o construir la interfaz con Java. No se acepta sustituir PostgreSQL por listas fijas o archivos.

5. Modelo de datos y registros iniciales

Cree la base de datos papeleria_campus y la tabla producto. Elabore los scripts de estructura y datos; el identificador lo genera PostgreSQL. Todos los campos, salvo el identificador automático, son obligatorios.

Campo	Tipo en PostgreSQL	Regla del ejercicio
id_producto	INTEGER	Clave primaria autogenerada (IDENTITY). No editable en el formulario.
codigo	VARCHAR(10)	NOT NULL y UNIQUE. De 1 a 10 caracteres; almacenar sin espacios laterales y en mayúsculas.
nombre	VARCHAR(80)	NOT NULL. De 3 a 80 caracteres tras quitar espacios laterales.
categoria	VARCHAR(20)	NOT NULL. Solo Papelería, Tecnología o Accesorios.
precio	NUMERIC(10,2)	NOT NULL y mayor que 0. Ingreso permitido: 0.01 a 99999999.99, con hasta 2 decimales.
stock	INTEGER	NOT NULL. Entero de 0 a 2147483647.

Restricciones: además de PRIMARY KEY, NOT NULL y UNIQUE, use CHECK para el código normalizado no vacío, la longitud del nombre, la categoría, el precio positivo y el stock no negativo. La interfaz debe rechazar precios con más de dos decimales antes de enviarlos a PostgreSQL. [5]

Datos de prueba obligatorios

Cargue exactamente estos 12 productos, sin filas adicionales, antes de ejecutar las pruebas de búsqueda. Los precios son ficticios, expresados en córdobas (C$); utilice el punto como separador decimal al ingresarlos.

Código	Nombre	Categoría	Precio C$	Stock
P001	Cuaderno universitario	Papelería	65.00	25
P002	Cuaderno cuadriculado	Papelería	72.50	12
P003	Lapicero azul	Papelería	12.00	80
P004	Resma de papel carta	Papelería	185.00	0
T001	Memoria USB 32 GB	Tecnología	210.00	8
T002	Mouse inalámbrico	Tecnología	320.00	6
T003	Teclado USB	Tecnología	450.00	4
T004	Cable HDMI	Tecnología	180.00	0
A001	Mochila escolar	Accesorios	850.00	5
A002	Estuche de lápices	Accesorios	95.00	18
A003	Botella reutilizable	Accesorios	240.00	0
A004	Regla metálica	Accesorios	45.00	20

Normalización: aplique trim a código y nombre, y convierta el código a mayúsculas antes de validar y guardar. Se permite stock = 0. Utilice BigDecimal para el precio en Java.

6. Comportamiento obligatorio

Interfaz y carga de información

El TableView debe mostrar seis columnas: ID, código, nombre, categoría, precio y stock. Configure TableColumn y una ObservableList<Producto> alimentada desde ResultSet. Al abrir o recargar, consulte PostgreSQL y muestre el contador de registros visibles. Orden inicial: código ascendente. [1, 3]

Incluya formulario para los cinco campos editables y botones Nuevo/Limpiar formulario, Guardar, Actualizar y Eliminar. Seleccionar una fila debe cargar sus valores y conservar su ID sin permitir su edición. Agregue Buscar, Limpiar filtros y Recargar; no confunda limpiar el formulario con quitar los filtros.

Búsqueda y filtros: pruebas B1 a B7

Use texto parcial, sin distinguir mayúsculas y minúsculas, sobre nombre O código; categoría mediante ComboBox con opción Todas; y CheckBox Solo con existencias (stock > 0). Combine los filtros con AND. Cada búsqueda debe consultar PostgreSQL, no solo una lista en memoria. [1, 2]

Caso	Entrada sobre los 12 datos iniciales	Resultado esperado
B1	Texto: CUADERNO; sin otros filtros.	P001 y P002; contador 2.
B2	Texto: T00; sin otros filtros.	T001 a T004; contador 4.
B3	Categoría: Papelería; texto vacío.	P001 a P004; contador 4.
B4	Solo con existencias; categoría Todas.	9 productos; excluye P004, T004 y A003.
B5	Texto USB + Tecnología + Solo con existencias.	T001 y T003; contador 2.
B6	Texto: ZZZ999; sin otros filtros.	0 filas, contador 0 y aviso Sin coincidencias.
B7	Pulsar Limpiar filtros después de B5.	Texto vacío, Todas, casilla desmarcada y 12 filas.

Operaciones CRUD y sincronización

Con filtros limpios: inserte X001, Producto de prueba, Accesorios, 10.00, stock 1 (13 filas). Actualice solo ese producto a precio 15.50 y stock 3 (13 filas). Cancele su eliminación (debe permanecer); luego confírmela (12 filas). Verifique cada cambio con SELECT y al reiniciar la aplicación.

Después de guardar, actualizar o eliminar con éxito, recargue desde PostgreSQL y respete los filtros vigentes: un producto que ya no cumpla el filtro debe desaparecer del listado. UPDATE y DELETE deben actuar por id_producto y afectar solo la fila seleccionada.

Acceso a datos

Separe Producto, un DAO (objeto de acceso a datos), controlador e interfaz. Use PreparedStatement para todos los valores variables, executeQuery para consultas y executeUpdate para escrituras. Cierre Connection, PreparedStatement y ResultSet con try-with-resources o una gestión equivalente verificable. Mantenga la configuración de conexión fuera del código y no publique contraseñas reales. [1, 4]

7. Validaciones y pruebas de error

Ejecute V1 a V9 al crear y al actualizar, cuando corresponda; V10 aplica a Actualizar y Eliminar. Un rechazo debe mostrar un mensaje concreto y dejar la base de datos sin cambios. Cada grupo V vale 0.5 puntos dentro de C6; se obtiene solo si pasan todos sus subcasos aplicables.

Grupo	Entradas o acción de prueba	Resultado exigido
V1	Código vacío, solo espacios o 11 caracteres.	Rechaza; exige de 1 a 10 caracteres tras trim.
V2	Nombre solo con espacios, AB o 81 caracteres.	Rechaza; exige de 3 a 80 caracteres tras trim.
V3	Categoría sin seleccionar; valor ajeno al catálogo si el control permite escribir.	Rechaza o impide introducir una categoría no permitida.
V4	Precio: abc.	Rechaza sin cerrar la aplicación.
V5	Precio: 0 y -1.	Rechaza ambos valores.
V6	Precio: 12.345 y 100000000.00.	Rechaza exceso de decimales y de rango; no redondea silenciosamente.
V7	Stock: abc y 2.5.	Rechaza texto y cantidades no enteras.
V8	Stock: -1 y 2147483648; luego stock 0.	Rechaza los dos primeros y admite 0.
V9	Crear otro producto con código " p001 "; actualizar otro producto a P001.	Rechaza duplicados tras normalizar. Editar P001 conservando su propio código sí debe permitirse.
V10	Actualizar o eliminar sin una fila seleccionada.	Bloquea la acción e informa que debe seleccionar un producto.

Excepciones y operaciones fallidas: E1 a E4

E1. Conexión: pruebe un puerto incorrecto en una configuración de prueba. Debe aparecer un aviso comprensible, la aplicación no debe cerrarse inesperadamente y el fallo no debe presentarse como una consulta exitosa con cero resultados.

E2. Restricción SQL: provoque una duplicidad o un CHECK fallido mediante una llamada de prueba al DAO, sin validación de interfaz. Debe gestionarse SQLException y conservarse la integridad. No desactive restricciones ni agregue botones de prueba a la interfaz final.

E3. Cero filas afectadas: seleccione un producto temporal, elimínelo directamente en la base de prueba e intente actualizarlo desde la aplicación. No debe anunciar éxito; debe advertir que ya no existe y permitir recargar. Aplique el mismo control a DELETE.

E4. Diagnóstico: registre la operación y el SQLState en consola o archivo. Muestre al usuario un mensaje sin contraseñas ni trazas técnicas. Una excepción no debe producir un aviso falso de guardado exitoso. [4]

Use exclusivamente una base local de pruebas. Al terminar cada bloque, restaure los 12 datos iniciales. Documente entrada, resultado esperado, resultado obtenido, estado y evidencia en pruebas.md; las capturas complementan, pero no sustituyen, el proyecto ejecutable.

8. Rúbrica analítica | 30 puntos

La calificación se obtiene sumando C1 a C8. Cada subcriterio se verifica por ejecución, consulta a PostgreSQL o revisión del código, según corresponda. Salvo los niveles parciales expresamente indicados, se asigna el valor completo si cumple y 0 si no cumple.

Parte A. Datos, acceso y consulta | 15 puntos

Criterio / valor	Reglas medibles de asignación y descuento
C1. Base de datos **IL2.3** **4 puntos**	1.0: estructura correcta: 0.5 por los seis campos y tipos especificados; 0.5 por clave primaria autogenerada. 1.0: 0.5 por NOT NULL en los cinco campos de entrada y 0.5 por UNIQUE en codigo. 1.0: cinco CHECK funcionales, a 0.2 cada uno: código normalizado no vacío, nombre, categoría, precio y stock. Se prueban también fuera de la interfaz. 1.0: exactamente los 12 registros iniciales correctos y sin adicionales; 0.5 si hay entre 6 y 11 correctos sin adicionales; 0 en otro caso. Deducción: el valor de cada verificación incumplida; máximo 4 puntos.
C2. JDBC y organización **IL2.2** **3 puntos**	Seis verificaciones de 0.5 cada una: a) La aplicación establece conexión real con PostgreSQL. b) Todos los valores variables de consultas y CRUD usan parámetros; ninguna entrada se concatena al SQL. c) Los recursos JDBC abiertos se cierran, también ante errores. d) Usa executeQuery y executeUpdate según la operación. e) El SQL reside en el DAO, no en el controlador de la interfaz. f) Conexión configurable externamente, con ejemplo sin secretos y sin contraseñas reales en los archivos versionados. Deducción: 0.5 por verificación fallida; máximo 3 puntos.
C3. TableView **IL2.2** **4 puntos**	1.0: las seis columnas muestran el campo correcto; 0.5 si solo 4 o 5 son correctas; 0 si son 3 o menos. 1.0: flujo comprobable ResultSet > Producto > ObservableList > TableView. No se sustituye con datos fijos. 1.0: 0.5 si la carga coincide con SELECT; 0.5 si una fila temporal agregada externamente aparece al pulsar Recargar. Retire esa fila tras la prueba. 1.0: 0.5 si la selección carga correctamente el formulario; 0.5 si el ID se conserva y no es editable. Deducción: el valor no obtenido; máximo 4 puntos.
C4. Búsqueda y filtros **IL2.2** **4 puntos**	B1, B2, B3 y B4: 0.5 por cada resultado y contador correctos (2.0). B5: 1.0 si combina los tres filtros y devuelve exclusivamente T001 y T003, con contador 2. B6: 0.5 si limpia las filas previas, muestra 0 y avisa Sin coincidencias. B7: 0.5 si restablece los controles y muestra las 12 filas. En cada prueba debe existir consulta a PostgreSQL; filtrar solo en memoria no acredita esa prueba. Deducción: valor de la prueba fallida; máximo 4 puntos.

Subtotal de la parte A: _____ / 15 puntos. La lectura del CRUD se valora en C3; la creación, actualización y eliminación se valoran en C5.

8. Rúbrica analítica | continuación

Parte B. CRUD, consistencia y evidencia | 15 puntos

Criterio / valor	Reglas medibles de asignación y descuento
C5. Operaciones CRUD **IL2.2** **6 puntos**	Crear, 2.0: 1.0 si inserta exactamente el producto indicado y persiste al reiniciar; 1.0 si el TableView y su contador se actualizan sin reiniciar. Actualizar, 2.0: 1.0 si modifica por ID solo el producto seleccionado y persiste; 1.0 si recarga el TableView y respeta los filtros vigentes. Eliminar, 2.0: 1.0 si solicita confirmación y Cancelar conserva el registro; 1.0 si Aceptar elimina exclusivamente la fila seleccionada y actualiza base, tabla y contador. Deducción: 1 punto por verificación no lograda; máximo 6. La consulta o lectura ya se valora en C3.
C6. Validaciones **IL2.3** **5 puntos**	V1 a V10: 0.5 por cada grupo de pruebas de la página 4. Para acreditar un grupo deben pasar todos sus subcasos aplicables, mostrarse un mensaje pertinente y evitarse cambios inválidos en PostgreSQL. Las pruebas se realizan al insertar y al actualizar, salvo V10, que se realiza al actualizar y eliminar. Deducción: 0.5 por grupo fallido, aunque falle más de una vez o en varios subcasos; máximo 5 puntos.
C7. Excepciones y consistencia **IL2.3** **2 puntos**	E1: 0.5 por aviso controlado de fallo de conexión, sin cierre inesperado ni falso listado exitoso. E2: 0.5 por gestionar SQLException de unicidad o CHECK en la capa de acceso, sin perder consistencia. E3: 0.5 por verificar filas afectadas en UPDATE y DELETE y no anunciar éxito cuando son cero. E4: 0.5 por registrar SQLState y la operación, mostrar un mensaje seguro y evitar avisos falsos de éxito ante excepciones. Deducción: 0.5 por prueba fallida; máximo 2 puntos.
C8. Documentación y Git **IL2.2 / IL2.3** **2 puntos**	0.5: README con versiones utilizadas, dependencias, creación de base de datos, configuración y pasos de ejecución que permiten reproducir el trabajo. 0.5: pruebas.md incluye B1-B7, el ciclo CRUD y V1-V10/E1-E4, con resultados obtenidos y evidencias; se registran también las pruebas fallidas. 0.5: GitHub accesible al docente y al menos tres commits no vacíos, con cambios reales y mensajes sobre búsquedas, validaciones e integración CRUD. 0.5: documenta el flujo interfaz > controlador > DAO > PostgreSQL y el regreso al TableView; declara fuentes o herramientas de apoyo utilizadas. Deducción: 0.5 por verificación incumplida; máximo 2 puntos.

TOTAL DE LA RÚBRICA: 30 PUNTOS

C1 4 + C2 3 + C3 4 + C4 4 + C5 6 + C6 5 + C7 2 + C8 2 = 30.

Subtotal parte A: ____ / 15 Subtotal parte B: ____ / 15 Total: ____ / 30

Las validaciones de interfaz (C6), las restricciones de la base (C1) y la gestión de excepciones SQL (C7) se comprueban por separado: superar una capa no reemplaza las otras.

9. Entrega y reglas de calificación

Archivos y evidencia

Entregue un archivo Apellido_Nombre_LabTableView.zip y el enlace del repositorio GitHub. El proyecto debe incluir src, recursos de interfaz y pom.xml o build.gradle, o una configuración equivalente documentada. No entregue solo el ejecutable, capturas o archivos .class.

Elemento	Contenido verificable
sql/estructura.sql	Creación de la tabla y restricciones. Documentar en README cómo crear y seleccionar papeleria_campus antes de ejecutarlo.
sql/datos.sql	Carga de los 12 productos iniciales. Indicar cómo restaurar la base de prueba.
README.md	Autor, versiones, dependencias, configuración, ejecución, organización de clases y fuentes de apoyo.
pruebas.md y evidencias/	Matriz de resultados y al menos tres evidencias legibles: listado inicial, filtro combinado y una validación. Incluir evidencia del ciclo CRUD.
Configuración de ejemplo	Archivo de ejemplo sin secretos o listado de variables de entorno. Excluir la configuración real mediante .gitignore.
Repositorio GitHub	Acceso de lectura para el docente y tres commits de avances reales. No es necesario hacerlo público.

Reglas de descuento

1. Descuentos técnicos. Se descuentan exclusivamente los valores publicados en C1-C8; ningún criterio queda por debajo de 0. Un mismo subcriterio fallido se descuenta una sola vez, aunque el error se repita. No se agrega una multa por un defecto ya valorado.

2. Verificación independiente. No hay cero global automático por un fallo de ejecución: se califican los criterios cuya evidencia pueda comprobarse. Las capturas no acreditan por sí solas conexión, persistencia o parametrización. No se penalizan fallos exclusivos del entorno del docente.

3. Atraso propuesto. El docente debe confirmar esta regla y publicar fecha, hora y zona horaria antes de asignar la tarea. Con plazo confirmado: descontar 1 punto por cada 24 horas de atraso o fracción, con un máximo de 3. Más de 0 y hasta 24 horas de atraso: -1; más de 24 y hasta 48: -2; más de 48: -3. Sin plazo publicado no se aplica descuento por atraso.

4. Autoría. El trabajo es individual en esta propuesta. Declare el código reutilizado y las herramientas de apoyo. Los posibles casos de integridad académica se tramitan conforme a la normativa institucional; no se inventan penalizaciones adicionales en esta rúbrica.

Calificación final = máximo(0, suma de C1 a C8 - descuento por atraso).
Ejemplo: 25 puntos en la rúbrica y 30 horas de atraso = 23/30, si la regla de atraso fue confirmada.

Referencias técnicas de apoyo

[1] pgJDBC: consultas, parámetros y ResultSet

[2] PostgreSQL: coincidencia de patrones e ILIKE

[3] OpenJFX: TableView y su lista de elementos

[4] Oracle: tratamiento de SQLException y cierre de recursos

[5] PostgreSQL: restricciones de integridad

Caso, datos, pruebas y rúbrica elaborados para este laboratorio. Las referencias fundamentan el uso de las tecnologías; no constituyen la solución del ejercicio. Consulta: 5 de octubre de 2026. Utilice y documente las versiones disponibles en su laboratorio.